Special Participation B: Kimi on HW12

Turn 1: Initial Acceptance

User: You are a Teaching Assistant for a graduate-level Deep Learning / Systems course, and you will rigorously, precisely, and correctly assist with coding exercises—but only when explicitly instructed. [full instructions]... Do you understand? Please reply yes or no, and ask any clarifying questions if the prompt is unclear.

Assistant: Yes, I understand the workflow and constraints.

To ensure optimal assistance, please clarify:

1. **Indexing scope**: When describing TODOs, should I also flag potential ambiguities or missing specifications in the assignment (e.g., unclear tensor shapes, unspecified edge cases)?

2. **LaTeX format**: For the code blocks, should I use a specific LaTeX package (`listings`, `minted`, etc.) or maintain the generic `verbatim` style for your compilation environment?

3. **Trace completeness**: For the full conversation trace, should I include intermediate analysis (e.g., "I see this function expects X") or only final outputs and user commands?

Standing by for file uploads.

Turn 2: sample_gaussian Task

User: Now, you will be implementing VAE: variational autoencoder. First, implement the TODO for the reparametrization trick in sample_gaussian:

```
import numpy as np
import os
import shutil
import torch
from models.vae import VAE
from torch.nn import functional as F
from torchvision import datasets, transforms

bce = torch.nn.BCEWithLogitsLoss(reduction='none')

################################################################################
# Please familiarize yourself with the code below.
#
# Note that the notation is
# argument: argument_type: argument_shape
#
# Furthermore, the expected argument_shape is only a guideline. You're free to
# pass in inputs that violate the expected argument_shape provided you know
# what you're doing
################################################################################

def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
```

```
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    #
        ################################################################################

    # TODO: Modify/complete the code here
    # Sample z
    #
        ################################################################################


    #
        ################################################################################

    # End of code modification
    #
        ################################################################################

    return z
```

Assistant:

```
def sample_gaussian(m, v):
    """
    Element-wise application reparameterization trick to sample from Gaussian

    Args:
        m: tensor: (batch, ...): Mean
        v: tensor: (batch, ...): Variance

    Return:
        z: tensor: (batch, ...): Samples
    """
    #
        ################################################################################

    # TODO: Modify/complete the code here
    # Sample z
    #
        ################################################################################

    epsilon = torch.randn_like(m)  # Sample epsilon ~ N(0, 1)
    z = m + torch.sqrt(v) * epsilon  # Reparameterization trick: z = mu + sigma *
        epsilon
    #
        ################################################################################

    # End of code modification
    #
        ################################################################################

    return z
```

“

V: Completely correct. Kimi one-shot this question. The only thing is Kimi used torch.randn_like(m) instead of v, but since they have the same shape it's correct.

Turn 3: negative_elbo_bound Task

User: Now, implement negative_elbo_bound here:

```
import torch
import utils as ut
from models import nns
from torch import nn
from torch.nn import functional as F

class VAE(nn.Module):
    def __init__(self, nn='v1', name='vae', z_dim=2):
        super().__init__()
        self.name = name
        self.z_dim = z_dim
        # Small note: unfortunate name clash with torch.nn
        # nn here refers to the specific architecture file found in
        # codebase/models/nns/*.py
        # models = getattr(nns, nn)
        self.enc = nns.Encoder(self.z_dim)
        self.dec = nns.Decoder(self.z_dim)

        # Set prior as fixed parameter attached to Module
        self.z_prior_m = torch.nn.Parameter(torch.zeros(1), requires_grad=False)
        self.z_prior_v = torch.nn.Parameter(torch.ones(1), requires_grad=False)
        self.z_prior = (self.z_prior_m, self.z_prior_v)

    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)
        #
            ################################################################################

        # TODO: Modify/complete the code here
        # Compute negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs should all be scalar
        #
            ################################################################################


        #
            ################################################################################

        # End of code modification
```

```
        #
            ##########################################################################

        return nelbo, kl, rec
```

Assistant:

```
    def negative_elbo_bound(self, x):
        """
        Computes the Evidence Lower Bound, KL and, Reconstruction costs

        Args:
            x: tensor: (batch, dim): Observations

        Returns:
            nelbo: tensor: (): Negative evidence lower bound
            kl: tensor: (): ELBO KL divergence to prior
            rec: tensor: (): ELBO Reconstruction term
        """
        mean, var = self.enc.encode(x)
        z = ut.sample_gaussian(mean, var)
        logits = self.dec.decode(z)
        #
            ##########################################################################

        # TODO: Modify/complete the code here
        # Compute negative Evidence Lower Bound and its KL and Rec decomposition
        #
        # Note that nelbo = kl + rec
        #
        # Outputs should all be scalar
        #
            ##########################################################################

        # Compute KL divergence between q(z|x) and p(z)
        kl = ut.kl_normal(mean, var, self.z_prior_m.expand_as(mean), self.
            z_prior_v.expand_as(var)).sum()

        # Compute reconstruction term (negative log-likelihood)
        rec = -ut.log_bernoulli_with_logits(x, logits).sum()

        # Compute negative ELBO
        nelbo = kl + rec
        #
            ##########################################################################

        # End of code modification
        #
            ##########################################################################

        return nelbo, kl, rec
```

X: Kimi struggled here. Kimi was able to identify the most important parts (like, using kl_normal and -log_bernoulli_with_logits, but it summed over them — not averaged. It was able to get that on a second try, though.